Syrian Private University

Faculty of Artificial Intelligence Engineering

Department of Artificial Intelligence Engineering and Data Science

وكيل ذكي لإدارة الميزانية العائلية قائم على استدعاء الأدوات

Family Budget Agent

Graduation Project 2 Report (Part 1), prepared to obtain the degree of B.Sc. in the Department of Artificial Intelligence Engineering and Data Science

Author

Ayham Al-Salem

Supervised by

Dr. Nasreen Suleiman

Eng. Zainab Baghdadi

دمشق: 1448-1449هـ

Damascus: 2026–2027

SUPERVISOR CERTIFICATION

I certify that the preparation of this project, entitled

"Family Budget Agent,"

prepared by

Ayham Al-Salem

was carried out under my supervision at the Faculty of Artificial Intelligence Engineering, Syrian Private University, in partial fulfillment of the requirements for the degree of B.Sc. in the Department of Artificial Intelligence Engineering and Data Science.

Name: Dr. Nasreen Suleiman

Date: ……

Signature: ……

Name: Eng. Zainab Baghdadi

Date: ……

Signature: ……

الملخّص

في مشروع التخرج الأول بُني مساعد للمصروفات باللهجة السورية كخط معالجة ثابت (pipeline): تحدّد الشيفرة جميع الخطوات مسبقاً، ولا ينفّذ النموذج اللغوي إلا خطوة واحدة منها، مثل استخراج المصروف من الرسالة. لذلك يسجّل مصروفاً واحداً على الأكثر من كل رسالة، ويجيب عن كل سؤال من الملخص نفسه للشهر الحالي، ولا يحتفظ بذاكرة للمحادثة، فلا يُربط ردّ المستخدم بسؤال سابق.

يقدّم هذا المشروع Family Budget Agent، الذي يحوّل ذلك المساعد إلى وكيل ذكي يستدعي الأدوات وفق منهجية ReAct. يقرّر الوكيل لكل رسالة الأدوات التي يحتاجها، ويستطيع تنفيذ عدة أدوات متتالية، ويطّلع على نتيجة كل أداة قبل أن يقرّر خطوته التالية. ويُجاب عن كل سؤال رقمي بأداة تحسب الرقم المطلوب بدقة عبر استعلامات SQL لأي فترة. وعندما يكون المبلغ أو الفئة أو العملة ناقصاً أو غامضاً يسأل الوكيل المستخدم ويربط ردّه بالطلب الأصلي، كما يسترجع تصحيحات المستخدم السابقة ويستخدمها أمثلةً للرسائل المشابهة دون إعادة تدريب النموذج. يعتمد الوكيل على نموذج Llama 3.3 70B عبر واجهة سحابية، ويُقارَن به النموذج الأصغر Llama 3.1 8B داخل الوكيل نفسه.

يعرض هذا الجزء الأول من التقرير المشكلة العلمية وأهداف المشروع، ويراجع ست دراسات حديثة تتناول الوكلاء المعتمدين على النماذج اللغوية، واستدعاء الأدوات، والاستعلامات المالية من قواعد البيانات، واستدعاء الأدوات باللغة العربية، وطلب التوضيح من المستخدم، والتعلّم من تعديلاته. كما يقدّم الخلفية النظرية، ويصف المعمارية المقترحة وأدواتها ومجموعة بيانات التقييم المخطط لها باللهجة السورية.

الكلمات المفتاحية: الوكلاء الذكيون، استدعاء الأدوات، نماذج اللغة الكبيرة، إدارة الميزانية العائلية، اللهجة السورية.

Abstract

In Graduation Project 1, a Syrian-dialect expense assistant was built as a fixed pipeline: the code defines every step in advance, and the language model performs only one of them, such as extracting an expense from a message. As a result, it records at most one expense per message, answers every question from the same summary of the current month, and keeps no memory of the conversation, so a user’s reply cannot be linked to an earlier question.

This project presents Family Budget Agent, which turns that assistant into a tool-calling agent based on the ReAct paradigm. For each message, the agent decides which tools to call, can execute several of them in sequence, and observes each result before choosing its next step. Each numerical question is answered by a tool that computes the exact number through SQL, for any period. When an amount, category or currency is missing or ambiguous, the agent asks the user and links the reply to the original request, and it retrieves the user's past corrections as examples for similar messages, without retraining the model. The agent uses Llama 3.3 70B through a cloud API, and the smaller Llama 3.1 8B is compared with it inside the same agent.

This first part of the report states the scientific problem and the objectives, and reviews six recent studies on LLM agents, function calling, financial database queries, Arabic function calling, clarification, and learning from user edits. It then presents the theoretical background and describes the proposed architecture, its tools and the planned Syrian-dialect evaluation dataset.

Keywords: LLM Agents, Tool Calling, ReAct, Large Language Models, Family Budget Management, Syrian Dialect.

Table of Contents

Chapter One: Introduction
- 1.1 Introduction
- 1.2 Scientific Problem
- 1.3 Research Objectives
- 1.4 Research Contribution
- 1.5 Beneficiaries
- 1.6 Report Organization

Chapter Two: Theoretical Background
- 2.1 Large Language Models
- 2.2 From Pipelines to LLM Agents
- 2.3 Function Calling (Tool Use)
- 2.4 Prompt Engineering and In-Context Learning
- 2.5 Embeddings, Vector Databases and Retrieval
- 2.6 Uncertainty Estimation and Clarification
- 2.7 Learning from User Feedback Without Retraining
- 2.8 Evaluation Metrics
- 2.9 Chapter Summary

Chapter Three: Literature Review
- 3.1 Related Studies
- 3.2 Modern Techniques and Frameworks in the Domain
- 3.3 Comparative Analysis
- 3.4 Research Gap and Proposed Innovation

Chapter Four: Research Methodology
- 4.1 Introduction
- 4.2 Graduation Project 1 Pipeline vs. Proposed Agent
- 4.3 Proposed System Architecture
- 4.4 Dataset
- 4.5 Tools and Technologies
- 4.6 Chapter Summary

References

[The evaluation methodology and experimental results (Chapter Five) and the conclusion (Chapter Six) will be added in the final report.]

List of Tables

Table 3.1: Comparative Analysis of Related Studies

Table 4.1: Graduation Project 1 Pipeline vs. Proposed Agent

Table 4.2: Tools of the Agent

Table 4.3: Database Schema Specification

Table 4.4: Planned Evaluation Dataset

List of Figures

Figure 4.1: Architecture of the Family Budget Agent

Figure 4.2: Agent Request-Handling Flowchart

List of Equations

Equation 2.1: Self-Attention Mechanism

Equation 2.2: ReAct Action Space

Equation 2.3: Cosine Similarity

Equation 2.4: Sampling-Based Confidence

Equation 2.5: Argument Exact Match

Equation 2.6: Precision, Recall and F1-Score

Equation 2.7: Cohen's Kappa

Glossary of Terms and Symbols

Term (English)	المصطلح بالعربية	Symbol / Abbr.
Large Language Model	نموذج لغوي كبير	LLM
AI Agent	وكيل ذكي	—
Pipeline	خط معالجة	—
Tool Calling / Function Calling	استدعاء الأدوات	FC
Reasoning and Acting	الاستدلال والتنفيذ	ReAct
Chain-of-Thought	سلسلة الأفكار	CoT
JSON Schema	مخطط JSON	—
Tool Argument	معامل الأداة	—
Argument Exact Match	التطابق التام للمعاملات	ArgEM
Task Success Rate	نسبة نجاح المهام	—
Clarifying Question	سؤال توضيحي	—
Correction Memory	ذاكرة التصحيحات	—
In-Context Learning	التعلّم ضمن السياق	ICL
System Prompt	تعليمات النظام	—
Retrieval-Augmented Generation	التوليد المعزّز بالاسترجاع	RAG
Embedding	التضمين	—
Vector Database	قاعدة بيانات متجهية	Vector DB
Cosine Similarity	تشابه جيب التمام	$\cos\theta$
Hallucination	الهلوسة	—
Rate Limit	حدّ عدد الطلبات	—
Cohen's Kappa	معامل كوهين كابا	κ
System Usability Scale	مقياس قابلية الاستخدام	SUS
Application Programming Interface	واجهة برمجة التطبيقات	API
Llama 3.3 70B / Llama 3.1 8B	نموذجا لاما بحجم 70 و8 مليار معامل	Llama
Groq API	واجهة برمجة جروك	Groq
BAAI General Embedding (M3)	نموذج تضمين متعدد اللغات	BGE-M3
ChromaDB	قاعدة بيانات متجهية مفتوحة المصدر	ChromaDB

Term (English)	المصطلح بالعربية	Symbol / Abbr.
SQLite	محرك قواعد بيانات علائقي مدمج	SQLite
FastAPI	إطار عمل لبناء الواجهات البرمجية بلغة بايثون	FastAPI
Streamlit	إطار عمل لبناء تطبيقات الويب التفاعلية	Streamlit
Docker / Docker Compose	منصة الحاويات البرمجية	Docker
Ollama	منصّة تشغيل النماذج محلياً	Ollama
Syrian Pound (old / new)	الليرة السورية (القديمة / الجديدة)	SYP

Chapter One: Introduction

This chapter introduces the research topic, defines the scientific problem, states the research objectives, highlights the contribution of the work, identifies the beneficiaries, and explains the organization of the report.

1.1 Introduction

Managing household expenses has become a daily challenge for families in Syria and the wider Arab region. Most budgeting applications rely on forms and work in English, while Arabic dialects remain underrepresented in large language models (LLMs) [1]. As a result, many families find it hard to keep track of their expenses.

In Graduation Project 1, we developed the Smart Family Budget Assistant (Mizan), which lets families record expenses by writing short sentences in the Syrian dialect, such as "دفعت 5000 بنزين" ("I paid 5000 for fuel") [2]. The system used Llama 3.3 70B through the Groq API to extract the amount, category and date, stored the data in SQLite and ChromaDB, and answered questions with retrieval-augmented generation (RAG). On the test sentences that did not fail with API errors, it reached 100% amount accuracy, 95.9% category accuracy and a macro F1-score of 96.8% [2].

However, that system followed a fixed pipeline: each message was assigned a single type, and hard-coded logic executed a single action for it. Recent research shows that a language model can act as an agent, deciding at run time which tools to call and using each tool's result to plan its next step [3]. Such agents can also ask the user when a request is unclear [4], and improve from the user's feedback without retraining [5].

This project, Family Budget Agent, transforms the system of Graduation Project 1 into a tool-calling agent. The agent understands requests in the Syrian dialect, decides which tools to call, executes several actions for a single message, computes all numbers through database queries, asks the user when information is missing, and learns from the user's corrections. The agent keeps using a cloud LLM, and the project also studies how the size of the model affects the agent's accuracy.

1.2 Scientific Problem

Graduation Project 1 showed that an LLM can extract expenses from Syrian-dialect text with high accuracy [2]. However, an analysis of that system reveals the following limitations:

- **A fixed pipeline with one action per message:** the LLM assigns each message a single type, such as an expense, a missing amount, a query, an advice request or general chat, and the code executes one predefined path. At most one expense is recorded per message, so a request such as "جبت خضرا بـ3000 ولحمة بـ20000، قلي إذا تجاوزت ميزانية الأكل" (record two expenses, then check the food budget) cannot be completed. Functions for editing and deleting expenses exist in the data layer, but no message type or interface element uses them.
- **Answers from a fixed summary:** for every question, the same SQL summary of the current month, with the budget alerts and semantically retrieved records, is passed to the LLM, which selects the numbers and writes the answer. No other period and no comparison between months is computed, so such questions cannot be answered from the database. In the deployed interface, a question about total spending was answered with the food category only, and a wrong budget percentage was used [2].

- **Limited asking and no conversation memory:** the system asks only when the amount is missing, and each message is processed independently, so the user's reply cannot be linked to the earlier message. When the category is unclear, as in "دفعت 50 لأبو أحمد" ("I paid 50 to Abu Ahmad"), it is chosen without asking the user, and falls back to the default category "متفرقات" when it does not match a known one. Expenses also have no currency field, so the system cannot tell whether an amount is in old or new Syrian pounds, which differ since the currency was redenominated by removing two zeros [6].
- **Unreliable use of the cloud API:** every message is sent to the free tier of the Groq API. Even with automatic retries, 62 of 160 expense sentences (38.8%) failed with timeout or rate-limit errors in the evaluation, and the reported accuracy and F1-score cover only the remaining sentences [2]. Moreover, although the data was stored locally, the text of every message was processed in the cloud, so the system was not fully private.
- **No learning from the user:** when the user corrects a mistake, the correction is not reused, so the same mistake can happen again.
- **A small evaluation set:** the test set contains 180 sentences, of which only 10 are queries and 5 are advice requests [2].

The central scientific problem addressed by this project is therefore: how can a family budget assistant be designed as a tool-calling agent that understands the Syrian dialect, completes multi-step requests with numbers computed from the database, asks the user when a request is unclear, and learns from the user's corrections? This problem is divided into three research questions:

- **RQ1:** Can a smaller cloud model (Llama 3.1 8B) approach the accuracy of Llama 3.3 70B in expense extraction and tool calling for Syrian-dialect requests, when both work inside the same agent?
- **RQ2:** Does answering numerical questions through question-specific SQL tools reduce answer errors compared with passing a fixed SQL summary to the LLM, as in Graduation Project 1?
- **RQ3:** How much does the error rate decrease when the agent retrieves the user's previous corrections and uses them as examples?

1.3 Research Objectives

This research aims to design, implement and evaluate a tool-calling agent for family budget management in the Syrian dialect. The main objective is to transform the fixed pipeline of Graduation Project 1 into an agent that achieves the following sub-goals:

- Design and implement an agent based on the ReAct paradigm that decides which tools to call, executes several tool calls for a single message, and observes each result before deciding the next step.
- Develop tools for recording, editing and deleting expenses, querying totals and breakdowns, and checking budgets, so that all numerical results are computed through SQL rather than generated by the language model.
- Add a clarification mechanism that asks the user when a required argument is missing or when the model's outputs are inconsistent, instead of guessing, and keeps the recent conversation so that the user's reply is linked to the original request.
- Build a correction memory that stores the user's corrections and retrieves similar ones for new messages, without retraining the model.

- Support amounts in the old Syrian pound, the new Syrian pound and the US dollar, and ask the user when the currency is ambiguous.
- Build a Syrian-dialect evaluation dataset of 300–400 expense sentences and 150–200 tool-calling cases, including multi-step requests, unclear requests and correction sequences, with its labels checked by a second annotator.
- Handle cloud API errors with timeouts and automatic retries, and count the requests that still fail in the evaluation instead of excluding them.
- Evaluate the agent using tool selection accuracy, argument accuracy, task success rate and latency; compare it with the system of Graduation Project 1; compare Llama 3.3 70B with Llama 3.1 8B; and conduct a user study with 10–15 families.
- Package the system as containerized services with Docker Compose for reproducible deployment.

1.4 Research Contribution

This project aims to contribute to Arabic natural language processing and financial AI in the following ways:

- It extends Graduation Project 1 from a fixed pipeline into a tool-calling agent for family budget management in the Syrian dialect, a combination that none of the reviewed studies addresses (Chapter Three).
- It introduces a Syrian-dialect evaluation dataset for expense extraction and tool calling, including multi-step requests, unclear requests and correction sequences.
- It compares answering numerical questions from a fixed summary passed to the LLM (Graduation Project 1) with answering them through question-specific SQL tools (this project), on the same questions.
- It measures how much retrieved user corrections reduce the agent's errors over time, and how model size affects an agent working in an Arabic dialect.

1.5 Beneficiaries

The proposed system is expected to benefit several groups:

- **Arabic-speaking families:** who need a simple assistant that understands their dialect, records several expenses in one message, and answers questions with correct numbers.
- **Researchers in Arabic NLP:** who can use the dataset and the evaluation of tool calling in the Syrian dialect as a baseline for further work.
- **Fintech developers in the region:** who can build on an architecture in which financial numbers are computed by tools rather than generated by the LLM.
- **Educational institutions:** who can use the project as a teaching case on the difference between LLM pipelines and LLM agents.

1.6 Report Organization

This report presents the first part of the project in four chapters. Chapter One has introduced the research, defined the problem, and stated the objectives. Chapter Two presents the theoretical background, including LLMs, agents, function calling, prompting, retrieval, uncertainty estimation and evaluation metrics. Chapter Three reviews six related studies and identifies the gap that this work aims to fill. Chapter Four describes the proposed methodology, the system architecture and the planned dataset. The final report will extend Chapter Four with the

implementation, and will add Chapter Five, which presents the evaluation methodology, the experimental results and the discussion, and Chapter Six, which presents the limitations, the conclusion and future work.

Chapter Two: Theoretical Background

This chapter presents the theoretical concepts on which the proposed agent is built, and which are needed during its development and evaluation. It begins with large language models, then explains the difference between pipelines and agents, function calling, prompting, embeddings and retrieval, uncertainty estimation, learning from user feedback, and the evaluation metrics.

2.1 Large Language Models

Large language models are neural networks based on the Transformer architecture, pre-trained on large text corpora to predict the next token [7]. The core operation of the Transformer is scaled dot-product self-attention:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k})\, V \qquad (Eq.\ 2.1)$$

where Q, K and V are the query, key and value matrices, and d_k is the dimension of the key vectors. After pre-training, an LLM can perform a new task from instructions and a few examples placed in its prompt, without changing its weights; this is called in-context learning [8].

LLMs are released in different sizes. Larger models are generally more capable, while smaller models are faster and cheaper to run. This project uses two models of the Llama family [9] through the Groq cloud API: Llama 3.3 70B, which was used in Graduation Project 1, and the smaller Llama 3.1 8B.

2.2 From Pipelines to LLM Agents

In a **pipeline**, the developer defines the sequence of steps in advance, and the language model only fills in a step, such as classifying a message or extracting its fields. In an **agent**, the language model itself decides at run time which action to take, observes the result of that action, and decides the next step until the task is complete. The key difference is who controls the flow: the code in a pipeline, the model in an agent.

Chain-of-thought prompting showed that LLMs can produce intermediate reasoning steps before an answer [10]. ReAct combines this reasoning with actions by extending the agent's action space with the space of language [3]:

$$\hat{A} = A \cup L \qquad (Eq.\ 2.2)$$

where A is the set of task actions (such as calling a tool) and L is the space of language. An action from L is a *thought*: it does not change the environment, but it updates the context to support future reasoning or acting. A task is therefore solved as a sequence of thought–action–observation steps. To keep the loop safe, an agent usually limits the number of steps, and stops when it produces a final answer.

2.3 Function Calling (Tool Use)

Function calling is the ability of an LLM to invoke external functions in response to a user request [11], [12]. Each tool is described by a schema that defines its name, its purpose, and its arguments with their types and required fields. Given a request and the available schemas, the model makes three decisions: whether a call is needed, which function to call, and which argument values to pass [13]. The model outputs a structured call, usually in JSON, which the system validates against the schema, executes, and returns to the model as an observation.

Function calling is evaluated by checking both the selected function and its arguments; the Berkeley Function Calling Leaderboard, for example, checks calls structurally and also tests serial calls, parallel calls and the ability to abstain [12]. A system can also let the model write SQL queries freely, but routing requests to a fixed set of vetted, parameterized functions trades flexibility for reliability [14]. Generating correct SQL remains difficult even for fine-tuned models [15].

2.4 Prompt Engineering and In-Context Learning

Without fine-tuning, an LLM is adapted to a task through its prompt [8]. In this project, the prompt has four parts: a system prompt that defines the agent's role, its rules and the expense categories; the schemas of the available tools; a few examples of correct tool calls; and the conversation context.

The examples are selected dynamically: for each message, the most similar past corrections are retrieved and placed in the prompt (Section 2.7). Generation settings also matter: a low temperature makes the output more deterministic, which suits structured tool calls.

2.5 Embeddings, Vector Databases and Retrieval

An embedding is a dense vector that represents a text, so that texts with similar meanings have nearby vectors. Closeness is usually measured with cosine similarity:

$$\cos(u, v) = (u \cdot v) / (\|u\| \cdot \|v\|) \qquad \text{(Eq. 2.3)}$$

This project uses BGE-M3, a multilingual embedding model that supports more than 100 languages and produces 1024-dimensional dense vectors [16]; as in Graduation Project 1, it runs locally through Ollama. The vectors are stored in ChromaDB, a lightweight vector database that runs locally and returns the nearest vectors to a query. Retrieval-augmented generation (RAG) adds retrieved information to the model's prompt to ground its answer [17]. In this project, retrieval has two roles: semantic search over expense descriptions, and retrieving the user's past corrections as examples for new messages.

2.6 Uncertainty Estimation and Clarification

An LLM can produce a confident output even when a request is ambiguous or incomplete. A practical way to estimate confidence is to sample several outputs and measure how often they agree with the most likely (greedy) output [18]:

$$\text{confidence}(x) = (1/n) \sum_i \mathbb{1}[\hat{y}_i = \hat{y}_{\text{greedy}}] \qquad \text{(Eq. 2.4)}$$

where $\hat{y}_i$ are n sampled outputs and $\hat{y}_{\text{greedy}}$ is the output at temperature 0. Cole et al. found this sampling-repetition score more reliable than the model's likelihood or self-verification for deciding when to abstain, especially on ambiguous questions [18]; agreement between samples is also the basis of self-consistency decoding [19]. In tool use, a second signal is structural: a required argument that is missing from the request. When either signal indicates uncertainty, the agent should ask the user a clarifying question instead of inventing a value [4].

2.7 Learning from User Feedback Without Retraining

A deployed model can be improved without changing its weights by storing user feedback and reusing it in the prompt. MemPrompt pairs the model with a growing memory of cases where it misunderstood the user, together with the user's clarification, and retrieves feedback from similar past cases to enhance the prompt for a new query [20]. CIPHER retrieves the preferences

learned from the k most similar past contexts and adds them to the prompt [5]. This approach adapts to each user, is cheap, and keeps the learned information readable.

2.8 Evaluation Metrics

Tool selection accuracy is the fraction of requests for which the agent calls the correct tools. **Argument exact match** measures whether the extracted arguments match the reference exactly [13]:

$$ArgEM = (1/N_{+}) \Sigma_i \mathbb{1}[\hat{a}_i \equiv a_i] \qquad (Eq.\ 2.5)$$

where N_{+} is the number of requests that require a call, and ≡ denotes equality after normalizing equivalent forms, such as Arabic-Indic and Western digits. **Task success rate** is the fraction of requests completed fully and correctly, including multi-step requests [21], [4]. **Clarification accuracy** is the fraction of unclear requests in which the agent asks the expected question [4]. The **API failure rate** is the fraction of requests that fail because of cloud API errors; these requests are counted as failures.

For expense categories, precision, recall and the F1-score are used, averaged over categories (macro-F1), as in Graduation Project 1 [2]:

$$Precision = TP / (TP + FP),\ Recall = TP / (TP + FN),\ F1 = 2PR / (P + R) \quad (Eq.\ 2.6)$$

Agreement between the two dataset annotators is measured with Cohen's kappa [22]:

$$\kappa = (p_o - p_e) / (1 - p_e) \qquad (Eq.\ 2.7)$$

where p_o is the observed agreement and p_e is the agreement expected by chance. Usability in the user study is measured with the System Usability Scale (SUS), which gives a score from 0 to 100 [23]. Latency is measured as the time from receiving a message to returning the answer.

2.9 Chapter Summary

This chapter has presented the theoretical pillars of the proposed agent: large language models and their sizes, the difference between pipelines and agents with the ReAct paradigm, function calling, prompting and in-context learning, embeddings and retrieval, uncertainty-based clarification, learning from user feedback without retraining, and the metrics used to evaluate the agent.

Chapter Three: Literature Review

This chapter reviews six recent studies that cover the main parts of the proposed agent, presents the techniques that recur in these studies, compares them with the proposed system, and identifies the research gap that this project addresses.

3.1 Related Studies

Six studies from 2023–2026 were selected because each covers one part of the proposed agent: the agent paradigm, function-calling models, the financial domain, Arabic function calling, asking the user for clarification, and learning from user feedback.

3.1.1 Study 1 — ReAct: Synergizing Reasoning and Acting in Language Models (2023)

This study, by Yao et al. (Princeton University and Google Research, ICLR 2023) [3], starts from the problem that reasoning and acting in large language models have mainly been studied separately. Reasoning alone is not grounded in the external world, which leads to hallucination and error propagation.

The researchers propose ReAct, in which the model generates reasoning traces and actions in an interleaved manner. Reasoning helps the model create and update its action plans, while actions let it gather information from external sources such as a Wikipedia API.

Each task is solved as a sequence of thought–action–observation steps, using PaLM-540B prompted with only a few examples. The approach was tested on HotpotQA, FEVER, ALFWorld and WebShop, and fine-tuning on 3,000 examples let PaLM-8B outperform all PaLM-62B prompting methods.

ReAct outperformed imitation and reinforcement learning methods by an absolute success rate of 34% on ALFWorld and 10% on WebShop. Hallucination caused 0% of its failures, compared with 56% for chain-of-thought. However, the interleaved structure caused more reasoning errors, and complex tasks need more examples than the input length allows.

3.1.2 Study 2 — TinyAgent: Function Calling at the Edge (2024)

This study, by Erdogan et al. (UC Berkeley, EMNLP 2024) [21], starts from the problem that large models are too big to run locally, so their inference runs in the cloud. This causes privacy issues, needs a stable connection and adds latency.

The researchers propose TinyAgent, a framework for training and deploying small language models that perform function calling on the device itself. As an application, they build a local Siri-like assistant for MacBook with 16 functions, such as email, calendar and notes.

They generated 80K training examples with GPT-4-Turbo and fine-tuned TinyLlama-1.1B and Wizard-2-7B with LoRA. They also added Tool RAG, which selects only the tools a query needs, and 4-bit quantization to make the models smaller and faster.

Fine-tuning raised the 1.1B model's success rate from 12.71% to 78.89%. The final models reached 80.06% (1.1B) and 84.95% (7B), above GPT-4-Turbo (79.08%), and the 1.1B model ran locally at only 0.68 GB. However, the authors note that the risk of hallucination remains, and that synthetic data may carry cultural biases.

3.1.3 Study 3 — FinStat2SQL: A Text2SQL Pipeline for Financial Statement Analysis (2025)

This study, by Nguyen et al. (National Economics University, Vietnam, INLG 2025) [15], starts from the problem that Text2SQL remains difficult for complex financial queries. Vietnam's local accounting standard (VAS) differs from IFRS, which adds more complexity to querying financial data.

The researchers propose FinStat2SQL, a multi-agent pipeline that lets users query financial statements in natural language. It combines large and small language models for entity extraction, SQL generation and self-correction.

They built a database of 200 Vietnamese companies, generated 12,400 synthetic SQL training samples with large models, and fine-tuned Qwen 2.5 Coder models (1.5B, 3B and 7B) with LoRA. The evaluation used about 300 real financial questions, and the researchers checked the overlap between the synthetic and evaluation data.

The fine-tuned 7B model reached 61.33% accuracy with under 4 seconds of latency, outperforming GPT-4o mini (59.14%) but not Llama 3.3 70B (65.69%), while the best model, Gemini 2.0 Flash Thinking, reached 72.03%. However, the study covers only large listed companies and depends on closed-source models such as GPT-4o mini.

3.1.4 Study 4 — AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems (2026)

This study, by Nacar et al. (Tuwaiq Academy, ArabicNLP 2026) [13], starts from the problem that Arabic tool use is still insufficiently evaluated, as most tool-calling benchmarks focus on English.

The researchers organize AISA-ArabicFC, the first shared task on Arabic function calling. For each Arabic request, the system must decide whether to call a tool, choose the correct function and extract its arguments, or otherwise abstain.

The dataset has 12,220 requests in five dialect groups, including Levantine (16.0%), across eight domains such as banking and finance, with 27 tools. Systems were evaluated by function-name accuracy and argument exact match, with a separate analysis per dialect.

Twenty-one teams submitted valid results, and a LoRA-adapted 4B model ranked first (0.9279), ahead of the zero-shot GPT-4o baseline (0.8195). Argument extraction, not tool selection, was the main challenge, and one team found Levantine to be its weakest dialect, with most failures caused by date-format and enumeration patterns. However, the task covers single-turn requests with only one call, so it does not test multi-step plans.

3.1.5 Study 5 — Learning to Ask: When LLM Agents Meet Unclear Instruction (2025)

This study, by Wang et al. (Renmin University of China and The Chinese University of Hong Kong, EMNLP 2025) [4], starts from the problem that tool-using agents assume user instructions are always clear. When information is missing, agents tend to invent the missing arguments, which can cause hallucinations and risks.

The researchers propose Ask-when-Needed (AwN), which makes the agent check each instruction before calling a tool, and ask the user whenever something is unclear.

From 1,000 real user queries, they found four types of unclear instructions: missing key information (56.0%), multiple references, errors, and requests beyond the tools. They built

NoisyToolBench with 200 such instructions, and an automatic evaluator (ToolEvaluator) that plays the role of the user.

Tested on 8 LLMs, AwN raised GPT-4o's rate of correct clarifying questions from 0.52 to 0.90 for instructions with missing information. However, the authors note that a big gap to perfect performance remains, and that the automatic evaluation is not fully accurate.

3.1.6 Study 6 — Aligning LLM Agents by Learning Latent Preference from User Edits (2024)

This study, by Gao et al. (Cornell University and Microsoft Research, NeurIPS 2024) [5], starts from the problem that users often edit an agent's responses to match their own preferences. Fine-tuning the model for every user is costly and hard to scale.

The researchers propose CIPHER, which learns a written description of the user's preference from their edits and adds it to the prompt, while the model itself stays unchanged.

For each new request, CIPHER retrieves the most similar past requests using embeddings and combines their learned preferences. It was tested on summarization and email writing with a GPT-4 simulated user over 200 rounds, measuring the edit distance of the user's changes.

CIPHER reduced user edits by 31% in summarization and 73% in email writing compared with no learning, and won 73.3% of human comparisons against the best baseline. However, a clear gap remains compared with knowing the true preference, and the main evaluation used a simulated user.

3.2 Modern Techniques and Frameworks in the Domain

Across the six studies reviewed above, several techniques recur:

- Interleaving reasoning with actions in a thought–action–observation loop [3], which other agent frameworks build on [4].
- Structured function calling with defined schemas, evaluated by function selection and exact argument match [13], or by matching the whole plan of calls [21].
- Using a large model to generate synthetic training data for a smaller model, and checking the data before use [21], [15].
- Parameter-efficient fine-tuning with LoRA to adapt smaller models to a task [21], [15], [13].
- Retrieving only the relevant tools or examples for each request, to shorten the prompt and guide the model [21], [15].
- Prompting the agent to ask the user before calling a tool when information is missing [4].
- Storing user feedback and retrieving it for similar future requests, without fine-tuning [5].
- Automatic evaluation with a simulated user or an LLM judge [4], [5], [15].

This project adopts the techniques that do not require fine-tuning: the reasoning-and-acting loop, structured function calling, retrieval of relevant examples, asking when needed, and retrieval of user corrections.

3.3 Comparative Analysis

Table 3.1 compares the six reviewed studies with the proposed system along several dimensions: domain, main technique, language, tool use, interaction with the user, and the best reported result.

Study	Domain	Technique	Language	Tool use	Asks / Learns	Best Result
[3] Yao '23	QA and decision making	ReAct prompting (PaLM-540B)	English	Yes	No / No	+34% success on ALFWorld
[21] Erdogan '24	Mac assistant	Fine-tuned small models (1.1B, 7B)	English	Yes	No / No	80.06% (1.1B) vs. 79.08% (GPT-4-Turbo)
[15] Nguyen '25	Financial statements	Multi-agent Text2SQL, LoRA	Vietnamese	SQL generation	No / No	61.33% (7B)
[13] Nacar '26	Services incl. banking	Arabic function calling (shared task)	Arabic, 5 dialect groups	Single call	Abstains / No	0.9279 (Qwen3-4B + LoRA)
[4] Wang '25	General APIs	Ask-when-Needed	English	Yes	Yes / No	A1: 0.52 → 0.90 (GPT-4o)
[5] Gao '24	Writing assistant	Retrieval of learned preferences	English	No	No / Yes	31%–73% fewer edits
Family Budget Agent (Ours)	Family budget	ReAct agent, SQL tools, clarification, correction memory	Arabic / Syrian	Yes, multi-step	Yes / Yes	To be measured

Table 3.1: Comparative Analysis of Related Studies

3.4 Research Gap and Proposed Innovation

The analysis of previous studies shows that each study focused on only one aspect of LLM agents. ReAct and TinyAgent built tool-using agents, but not for Arabic or for finance. FinStat2SQL worked on finance, but on company financial statements rather than household expenses.

AISA-ArabicFC evaluated Arabic tool calling, but only single requests with one call. Learning to Ask and CIPHER improved how agents deal with users, by asking questions and learning from edits, but outside Arabic and outside the financial domain. Therefore, there is still a lack of systems that combine:

- A tool-calling agent that can plan several actions for a single request
- Arabic dialects, including the Syrian dialect
- Family budget and expense management
- Asking the user when a request is unclear

- Learning from the user's corrections over time

The previous studies also show several important ideas that can be used in developing the Family Budget Agent:

First, ReAct shows that combining reasoning with tool actions reduces hallucination (0% of its failures vs. 56% for chain-of-thought). **In our project**, it is the basis of the agent loop that replaces the fixed pipeline of Graduation Project 1.

Second, TinyAgent shows that function-calling accuracy depends strongly on how a model is adapted to the task: fine-tuning raised its 1.1B model from 12.71% to 78.89% success, and Tool RAG brought it to 80.06%, above GPT-4-Turbo (79.08%). **In our project**, the cloud models are used without fine-tuning, but a large and a smaller model are compared inside the same agent, and success is measured over the whole sequence of tool calls, as TinyAgent measures its whole plan.

Third, FinStat2SQL shows that financial questions can be answered from a database through SQL, that model size matters (its fine-tuned 7B model reached 61.33%, while Llama 3.3 70B reached 65.69%), and that example data must be checked against the evaluation data. **In our project**, totals and budgets are computed by SQL inside the tools, and the examples placed in the prompt are checked for overlap with the human-written test set.

Fourth, AISA-ArabicFC shows that argument extraction, not tool selection, is the main challenge in Arabic function calling, and one team traced most of its Levantine errors to dates and enumerated values. **In our project**, we evaluate both tool selection and argument accuracy, especially for amounts, dates and categories in the Syrian dialect.

Fifth, Learning to Ask shows that prompting the agent to ask raised GPT-4o's correct clarifying questions from 0.52 to 0.90. **In our project**, the agent asks when the amount, category or currency is unclear, and our unclear test cases follow the same four types.

Finally, CIPHER shows that retrieving what was learned from similar past interactions reduces user edits by 31% to 73% without fine-tuning. **In our project**, user corrections are stored in ChromaDB and retrieved for similar future messages.

Based on these results, our project will combine these techniques in one agent able to:

- Understand expense messages and questions in the Syrian dialect and decide which tools to call
- Execute several tool calls for a single message (record expenses, compute totals, check budgets)
- Compute all numbers through SQL queries instead of generating them with the language model
- Ask the user for clarification when the amount, category or currency is unclear
- Store the user's corrections and retrieve similar ones to improve future answers
- Compare Llama 3.3 70B with the smaller Llama 3.1 8B, and count failed API requests instead of excluding them

Chapter Four: Research Methodology

This chapter presents the proposed methodology. It first compares the pipeline of Graduation Project 1 with the proposed agent, then describes the system architecture and its components, and the planned dataset.

4.1 Introduction

The methodology follows an engineering-oriented research approach, as in Graduation Project 1 [2]. The proposed solution keeps the interface, the database and the embedding model of Graduation Project 1, and replaces its fixed pipeline with a tool-calling agent built on a cloud LLM. Each component is designed to be independently testable, so that the agent can be evaluated step by step.

4.2 Graduation Project 1 Pipeline vs. Proposed Agent

Table 4.1 summarizes how the proposed agent differs from the system of Graduation Project 1.

Aspect	Graduation Project 1 (pipeline)	Graduation Project 2 (agent)
Who decides the flow	The code, in advance	The LLM, at run time
Actions per message	One; at most one expense recorded	Several, in sequence
Uses the result of its actions	No	Yes, to decide the next step
Source of numbers	One SQL summary of the current month, read by the LLM	A SQL tool for each question, for any period
Missing or unclear information	Asks only for a missing amount; other details are filled in without asking	Asked of the user
Conversation memory	None; each message is processed alone	Recent conversation kept in the context
Editing and deleting	Not available in the chat or the interface	Through the chat, after confirmation
Currency	Syrian pound only	Old SYP, new SYP and USD
User corrections	Not reused	Stored and retrieved for similar messages
Adding a new capability	A new code path	A new tool definition
Language model	Llama 3.3 70B via Groq	Llama 3.3 70B via Groq; Llama 3.1 8B for comparison

Aspect	Graduation Project 1 (pipeline)	Graduation Project 2 (agent)
API errors	Retried, but failed sentences excluded from the reported accuracy	Retried; remaining failures counted

Table 4.1: Graduation Project 1 Pipeline vs. Proposed Agent

A simple test separates the two designs: if the full execution path of every message can be drawn before the model runs, the system is a pipeline; if the path depends on the model's decisions and on the tools' results, it is an agent.

4.3 Proposed System Architecture

The proposed system follows the architecture illustrated in Figure 4.1. It consists of an interface layer, an agent core, a cloud LLM layer, a set of tools, a data layer, and an offline evaluation component.

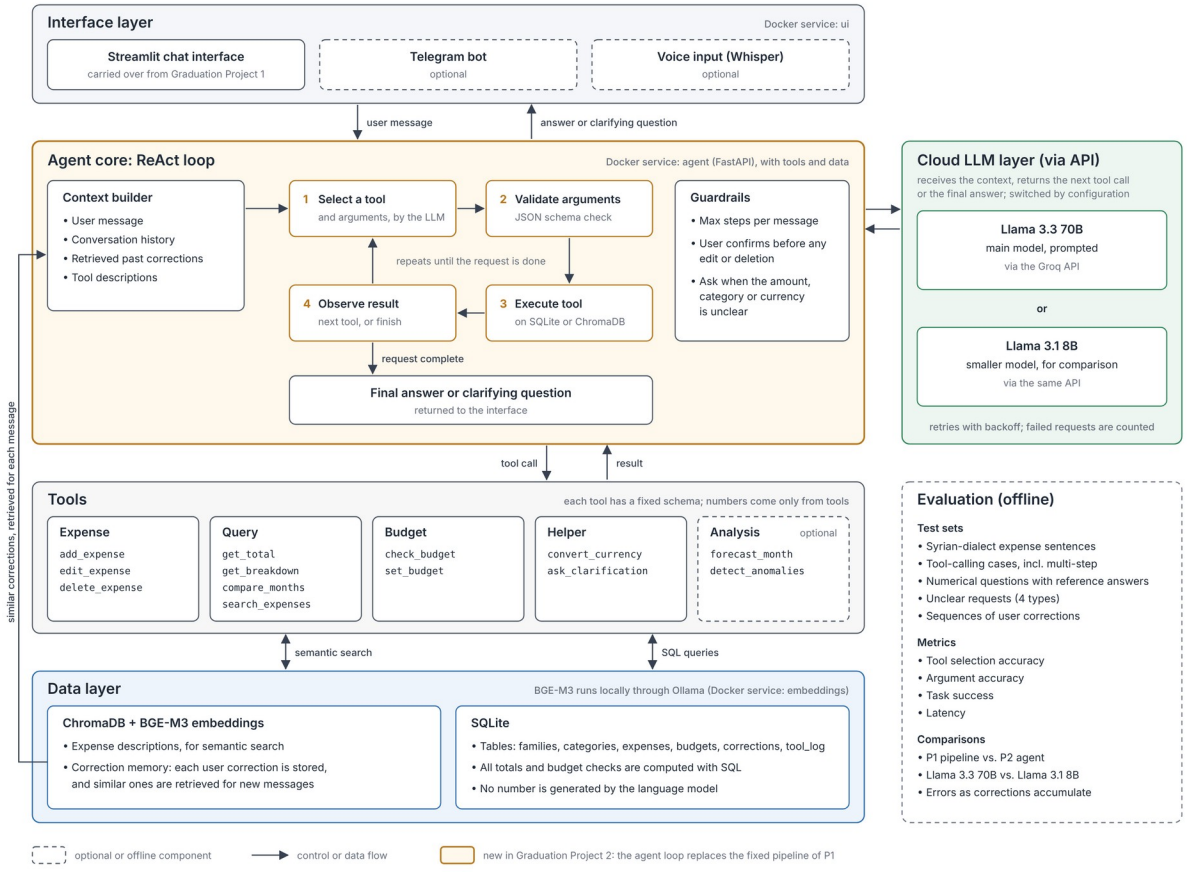


Figure 4.1: Architecture of the Family Budget Agent

- **Interface layer:** the Streamlit chat interface from Graduation Project 1; a Telegram bot and voice input are optional additions.
- **Agent core:** the ReAct loop that builds the context, lets the LLM choose a tool, validates and executes the call, and observes the result, with guardrails around it.

- **Cloud LLM layer:** Llama 3.3 70B as the main model and Llama 3.1 8B for comparison, both accessed through the Groq API behind one interface.
- **Tools:** fixed functions for expenses, queries, budgets, currency and clarification, plus two optional analysis tools.
- **Data layer:** SQLite for all structured records and numbers, and ChromaDB with BGE-M3 embeddings for semantic search and the correction memory.
- **Evaluation (offline):** the test sets, metrics and comparisons used in the experiments.

The system runs as three Docker services managed by Docker Compose: the interface, the agent (a FastAPI service that contains the agent core, the tools and the data layer), and the embedding server (Ollama running BGE-M3).

4.3.1 The Agent Loop

Figure 4.2 shows how the agent handles a single message. For each message, the agent performs the following steps:

- **Step 1 — Build the context:** the message, the recent conversation, the tool descriptions, and the most similar past corrections retrieved from the correction memory.
- **Step 2 — Select a tool:** the LLM returns a tool call with its arguments, a clarifying question, or the final answer.
- **Step 3 — Validate:** the arguments are checked against the tool's JSON schema; an invalid call is returned to the model as an error so it can correct it.
- **Step 4 — Confirm:** before any edit or deletion, the user is asked to confirm.
- **Step 5 — Execute and observe:** the tool runs on SQLite or ChromaDB, and its result is returned to the model as an observation.
- **Step 6 — Repeat or finish:** steps 2–5 repeat until the model produces the final answer, or until a fixed maximum number of steps is reached.

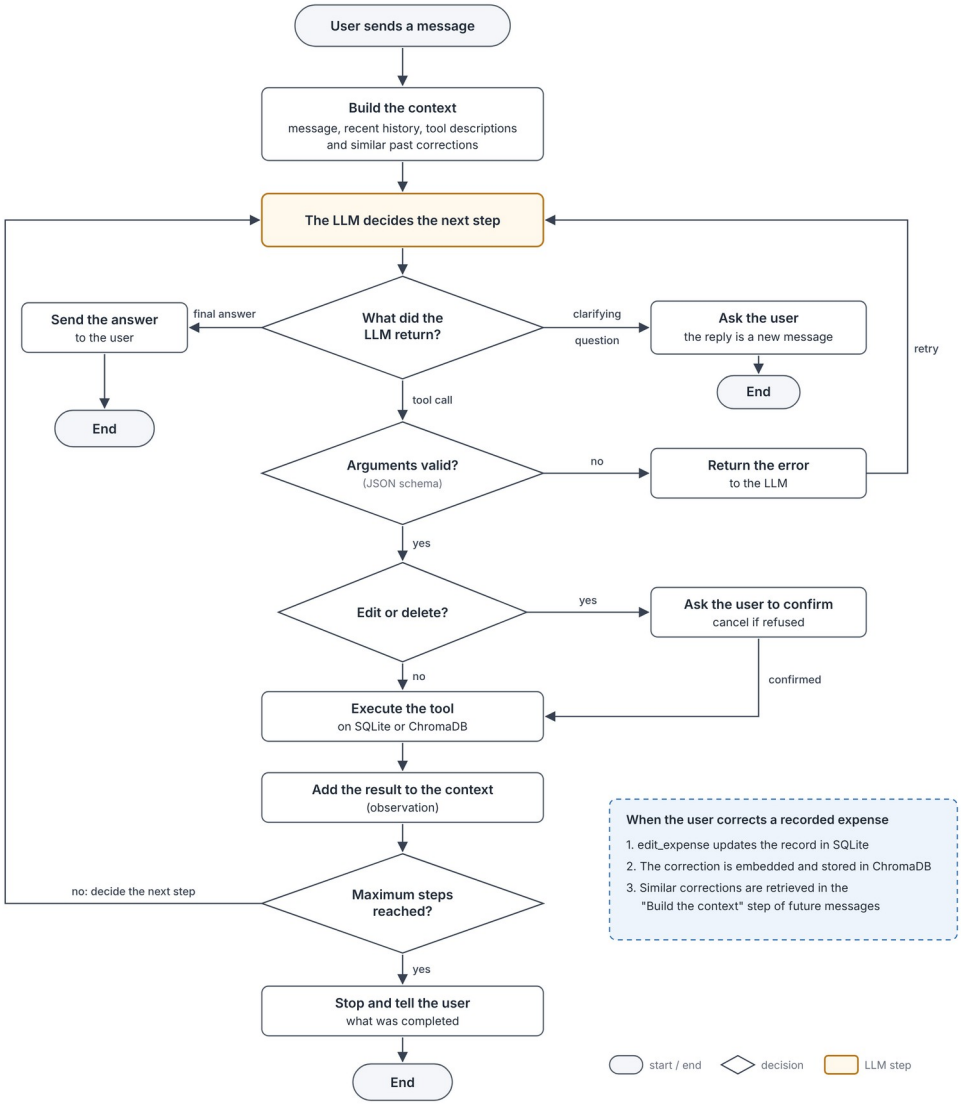


Figure 4.2: Agent Request-Handling Flowchart

For example, the message "جبت خضرا بـ3000 ولحمة بـ20000، قلي إذا تجاوزت ميزانية الأكل" is handled with three tool calls, after which the answer is written from the result of the last one:

```
add_expense(amount=3000, category="أكل وشرب")
add_expense(amount=20000, category="أكل وشرب")
check_budget(category="أكل وشرب", period="this_month")
```

The agent is implemented with direct function calling in Python, without a heavy agent framework, so that every step can be traced and evaluated.

4.3.2 Tools

Table 4.2 lists the tools available to the agent. The expense categories are the same 12 categories used in Graduation Project 1 [2].

Tool	Purpose	Main Arguments	Data Source
add_expense	Record a new expense	amount, currency, category, date, description	SQLite, ChromaDB
edit_expense	Correct a recorded expense, after confirmation	expense_id, fields to change	SQLite
delete_expense	Delete an expense, after confirmation	expense_id	SQLite
get_total	Total spending for a period or category	period, category (optional)	SQLite
get_breakdown	Spending per category	period	SQLite
compare_months	Compare spending between two months	month_a, month_b, category (optional)	SQLite
search_expenses	Find past expenses by meaning	query, period (optional)	ChromaDB
check_budget	Spending vs. limit, and the remaining amount	category, period	SQLite
set_budget	Set a monthly limit for a category	category, amount, month	SQLite
convert_currency	Convert between old SYP, new SYP and USD	amount, from_currency, to_currency	SQLite
ask_clarification	Ask the user a question	question, missing_fields	None
forecast_month (optional)	Expected spending for the next month (moving average)	category (optional)	SQLite
detect_anomalies (optional)	Unusual expenses or categories (z-score)	period	SQLite

Table 4.2: Tools of the Agent

4.3.3 Clarification Mechanism

The agent asks the user instead of guessing in two cases. First, when a required argument is missing from the request, such as the category in "دفعت 50 لأبو أحمد". Second, when the model's sampled outputs disagree: the model is sampled n times, the agreement with the greedy output is computed with Eq. 2.4 [18], and below a threshold the agent asks. Because every sample is an extra API call, n = 3 and n = 5 will be compared in both accuracy and the number of calls. Requests that no tool can serve, such as transferring money, are declined with an explanation, following the four types of unclear instructions in [4].

Currency needs special handling, because 1 new Syrian pound equals 100 old pounds [6]. When an amount could plausibly be in either currency, the agent asks, and the user's answer is stored as a correction for similar future messages.

4.3.4 Correction Memory

When the user corrects the agent, for example by changing the category, the amount or the currency, the system stores the original message, the wrong call and the corrected call. The message is embedded with BGE-M3 and saved in ChromaDB, and the correction is also kept in the corrections table in SQLite. For each new message, the agent retrieves the k most similar corrections by cosine similarity (Eq. 2.3) and adds them to the prompt as examples, following MemPrompt [20] and CIPHER [5]. The model itself is never retrained.

4.3.5 Cloud LLM Layer

The agent calls the LLM through one interface, so the model can be changed by configuration. The main model is Llama 3.3 70B, as in Graduation Project 1, and the comparison model is Llama 3.1 8B. Both are accessed through the Groq API with prompting only, without fine-tuning, and a low temperature is used for tool calls.

Following the lessons of Graduation Project 1, every request has a timeout and up to three automatic retries with exponential backoff, a tier with higher rate limits is used during the evaluation if needed, and any request that still fails is counted as a failure. The data is stored locally in SQLite and ChromaDB, but the text of each message is processed by the cloud API, so the system does not claim full privacy.

4.3.6 Database Design

The relational database is implemented in SQLite via SQLAlchemy. It keeps the four tables of Graduation Project 1 [2], adds the currency of each expense, and adds two new tables, as summarized in Table 4.3.

Table	Purpose	Key Fields
families	Family profile and monthly income	id, name, monthly_income, created_at
categories	The 12 expense categories	id, name_ar, name_en, budget_percent, keywords
expenses	Individual expense records	id, family_id, category_id, amount, currency, date, source_text, description
budgets	Monthly budget limits per category	id, family_id, category_id, monthly_limit, month
corrections (new)	User corrections reused by the agent	id, family_id, message, wrong_call, corrected_call, created_at
tool_log (new)	Every tool call, for evaluation and debugging	id, message_id, tool_name, arguments, result, status, latency_ms

Table 4.3: Database Schema Specification

4.4 Dataset

The dataset of Graduation Project 1, with 180 Syrian-dialect sentences of which 160 are expense sentences, is the starting point [2]. Because the agent must handle multi-step requests, unclear requests and user corrections, a larger and more varied dataset will be built. Since the model is not fine-tuned, no training data is needed: the dataset is used only for evaluation, and a few separate examples are written for the prompt. Arabic arguments need explicit testing: tool-

calling models make more argument errors on Arabic requests, often writing the arguments in English [24].

Part	Target Size	Content
Expense sentences	300–400	Syrian-dialect expenses across the 12 categories (about 25–30 per category), including the 160 expense sentences of Graduation Project 1, several expenses in one sentence, spelling mistakes, numbers written in words, and amounts in old SYP, new SYP and USD
Tool-calling cases	150–200	Requests with their expected tool calls and arguments, including multi-step requests
Unclear requests	About 50, within the tool-calling cases	The four types of [4]: missing information, multiple references, errors, and requests beyond the tools, each with the expected clarifying question
Numerical questions	At least 100	Questions about totals, breakdowns, comparisons and budgets, with reference answers computed from a fixed database of synthetic family expenses generated by a script
Correction sequences	Up to 50 corrections per sequence	Series of messages, built from the test sentences, in which a simulated user corrects the agent, as in CIPHER [5]

Table 4.4: Planned Evaluation Dataset

The dataset will be built in the following steps:

- **Annotation guidelines:** definitions of the categories, tools, arguments and currency rules, and the expected clarifying question for each type of unclear request.
- **Human-written sentences:** written by native Syrian speakers about everyday family expenses, together with the sentences of Graduation Project 1. An LLM may be used to suggest additional variations, but every sentence is reviewed by a native speaker.
- **Unclear requests:** created by modifying clear requests into the four types, following the construction of NoisyToolBench [4].
- **Test set:** written by people only, fixed before any experiment is run, and checked for overlap with the examples used in the prompt, following FinStat2SQL [15].
- **Annotation quality:** one annotator labels all the data, and a second annotator independently labels a random sample of 20–30%. Their agreement is measured with Cohen's kappa (Eq. 2.7) [22] with a target of $\kappa \geq 0.8$, and disagreements are resolved by discussion.

The Levantine part of the AISA-ArabicFC dataset (1,961 requests) [13] may also be used as an additional test of general Arabic function calling, after its license is checked. It covers different domains from family budgeting, so it complements the project's own test set rather than replacing it.

4.5 Tools and Technologies

- **Programming and services:** Python, FastAPI, Streamlit

- **Data:** SQLite with SQLAlchemy, ChromaDB, BGE-M3 embeddings served by Ollama
- **Language models:** Llama 3.3 70B and Llama 3.1 8B through the Groq API
- **Deployment:** Docker and Docker Compose

4.6 Chapter Summary

This chapter has presented the proposed methodology: the differences between the pipeline of Graduation Project 1 and the proposed agent, the architecture and its components (the agent loop, the tools, the clarification mechanism, the correction memory, the cloud LLM layer and the database), and the planned dataset. The final report will extend this chapter with the implementation details, and will present the evaluation methodology and the results in Chapter Five.

References

[1] B. Mousi, N. Durrani, F. Ahmad, M. A. Hasan, M. Hasanain, T. Kabbani, F. Dalvi, S. A. Chowdhury, and F. Alam, "AraDiCE: Benchmarks for Dialectal and Cultural Capabilities in LLMs," in Proc. COLING, 2025.

[2] A. Al-Salem, "Smart Family Budget Assistant Powered by LLM and RAG," Graduation Project 1 report, Faculty of Artificial Intelligence Engineering, Syrian Private University, Damascus, 2026.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023.

[4] W. Wang, J. Shi, Z. Ling, Y.-K. Chan, C. Wang, C. Lee, Y. Yuan, J. Huang, W. Jiao, and M. R. Lyu, "Learning to Ask: When LLM Agents Meet Unclear Instruction," in Proc. EMNLP, 2025, pp. 21773–21784.

[5] G. Gao, A. Taymanov, E. Salinas, P. Mineiro, and D. Misra, "Aligning LLM Agents by Learning Latent Preference from User Edits," in Proc. NeurIPS, 2024.

[6] CNBC Arabia, "Syria launches its new currency without two zeros: what are the details?" (in Arabic), 28 Dec. 2025. [Online]. Available: https://www.cnbcarabia.com/146997/2025/28/12/%D8%B3%D9%88%D8%B1%D9%8A%D8%A7-%D8%AA%D8%B7%D9%84%D9%82-%D8%B9%D9%85%D9%84%D8%AA%D9%87%D8%A7-%D8%A7%D9%84%D8%AC%D8%AF%D9%8A%D8%AF%D8%A9-%D8%AF%D9%88%D9%86-%D8%B5%D9%81%D8%B1%D9%8A%D9%86..-%D9%85%D8%A7%D8%B0%D8%A7-%D9%81%D9%8A-%D8%A7%D9%84%D8%AA%D9%81%D8%A7%D8%B5%D9%8A%D9%84%D8%9F

[7] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[8] T. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901, 2020.

[9] Llama Team, AI @ Meta, "The Llama 3 Herd of Models," arXiv:2407.21783, 2024.

[10] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in Advances in Neural Information Processing Systems, vol. 35, 2022.

[11] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[12] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, "The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models," in Proc. ICML, 2025.

[13] O. Nacar, W. Alzubaidi, and M. Alkhalifah, "AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems," in Proc. ArabicNLP, vol. 2: Shared Tasks, 2026, pp. 1531–1546.

[14] J. Kim, Y. Kim, Y. Lee, and H. Byun, "FinAI Data Assistant: LLM-based Financial Database Query Processing with the OpenAI Function Calling API," in Proc. CIKM, 2025, arXiv:2510.14162.

[15] Q. H. Nguyen, P. A. Trinh, P. Q. H. Mai, and T. P. Trinh, "FinStat2SQL: A Text2SQL Pipeline for Financial Statement Analysis," in Proc. INLG, 2025, pp. 449–464.

[16] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv:2402.03216, 2024.

[17] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[18] J. R. Cole, M. J. Q. Zhang, D. Gillick, J. M. Eisenschlos, B. Dhingra, and J. Eisenstein, "Selectively Answering Ambiguous Questions," in Proc. EMNLP, 2023, pp. 530–543.

[19] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in Proc. ICLR, 2023.

[20] A. Madaan, N. Tandon, P. Clark, and Y. Yang, "MemPrompt: Memory-assisted Prompt Editing with User Feedback," in Proc. EMNLP, 2022, pp. 2833–2861.

[21] L. E. Erdogan, N. Lee, S. Jha, S. Kim, R. Tabrizi, S. Moon, C. Hooper, G. Anumanchipalli, K. Keutzer, and A. Gholami, "TinyAgent: Function Calling at the Edge," in Proc. EMNLP: System Demonstrations, 2024, pp. 80–88.

[22] J. Cohen, "A Coefficient of Agreement for Nominal Scales," Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960.

[23] J. Brooke, "SUS: A 'Quick and Dirty' Usability Scale," in Usability Evaluation in Industry, P. W. Jordan et al., Eds. London: Taylor & Francis, 1996, pp. 189–194.

[24] A. Ersoy, E. Altinisik, K. M. Darwish, and H. T. Sencar, "Tool Calling for Arabic LLMs: Data Strategies and Instruction Tuning," in Proc. ArabicNLP, 2025, pp. 347–358.